

TP1 - Recorte consciente de audio

Peter Ahn - Lucas Dayan - Federico Del Cuadro
Universidad Torcuato Di Tella

I. INTRODUCCIÓN Y MARCO TEÓRICO

El *Recorte Consciente* del contenido es una técnica que permite acortar una grabación de audio a una duración objetivo preservando su estructura interna (estrofas, estribillos, cierres) en lugar de comprimir o truncar el material de forma arbitraria. A diferencia de los enfoques más tradicionales, el truncado simplemente descarta el final de la pieza, eliminando justamente su cierre, mientras que la aceleración uniforme de la reproducción distorsiona el tempo y la altura de todo el material, dejando de ser aceptable para el oído por encima de cierto factor de compresión.

Frente a estas limitaciones, el *recorte consciente* propone un enfoque distinto: en lugar de comprimir la pieza entera, se descartan fragmentos completos y se unen los extremos restantes, aprovechando que la música tiende a repetirse. Si el punto de unión cae en un lugar donde el material que sigue es muy similar al que debería haber sonado originalmente, el corte resulta inaudible para el oyente.

Este informe tiene como objetivo analizar distintos diseños de algoritmos que permiten resolver el problema del recorte consciente de audio, incluyendo técnicas de fuerza bruta, backtracking con podas de factibilidad y optimalidad, y programación dinámica. Las implementaciones se realizan tanto en C++ como en Python, lo cual permite comparar tanto el rendimiento teórico de cada algoritmo como su eficiencia práctica.

-
- El informe se organiza por secciones de la siguiente manera: en la Sección II se describe formalmente el problema y en la Sección III se presenta su modelado.
 - En las Secciones IV, V y VI se desarrollan los algoritmos de Fuerza Bruta, Backtracking y Programación Dinámica, respectivamente, junto con sus correspondientes análisis de complejidad. En la Sección VII se comparan las distintas implementaciones, mientras que en la Sección VIII se detallan las instrucciones de compilación y ejecución.
 - Luego, en la Sección IX se presenta la metodología de experimentación y en la Sección X se exponen y analizan los resultados obtenidos. Finalmente, en la Sección XI se presentan las conclusiones del trabajo.
-

II. DESCRIPCIÓN DEL PROBLEMA

El problema del recorte consciente consiste en seleccionar, dentro de una grabación, un subconjunto de pulsos que preserve el inicio y el final de la pieza, minimizando la discontinuidad audible introducida en cada punto donde se descartan pulsos intermedios.

Una grabación se modela como una secuencia de n pulsos, donde cada pulso i tiene asociado un vector de características $f_i \in R^d$ (d indica la cantidad de características por pulso) que resume su contenido armónico y tímbrico en ese instante. Una selección de k pulsos es una secuencia de índices

$$1 = i_1 < i_2 < \dots < i_k = n$$

Es decir, el primer y el último pulso se conservan siempre, de manera que el recorte comienza y termina donde lo hace la grabación original. Reproducir el pulso i y saltar a continuación al pulso j tiene un costo

$$c(i, j) = ||f_{i+1} - f_j||$$

que mide cuán distinto es el pulso al que efectivamente se salta respecto del siguiente pulso que debería haber sonado. En particular, $c(i, j) = 0$ cuando $j = i+1$. El costo total de una selección se define como

$$C(S) = \sum_{t=1}^{k-1} c(i_t, i_{t+1})$$

El objetivo del problema es encontrar la selección de k pulsos de costo mínimo.

El archivo de entrada tiene el siguiente formato: la primera línea contiene tres enteros n, d, k , indicando respectivamente la cantidad de pulsos de la grabación, la cantidad de características por pulso, y la cantidad de pulsos a conservar.

Ejemplo.

Con $n = 6, d = 1, k = 4$ y características $f = (0, 1, 5, 6, 2, 3)$

La selección óptima es 1 2 4 6, con costo $0,0 + 1,0 + 1,0 = 2,0$.

Las selecciones 1 2 3 6 y 1 2 5 6 cuestan ambas 3,0, lo cual muestra que no cualquier salto es igual de 'barato'.

La salida es una línea con los k enteros $i_1, \dots, i_k$ (indexados desde 1) en orden creciente, indicando los pulsos conservados por la selección óptima.

III. MODELADO

Como los extremos están fijos, elegir una solución es elegir un subconjunto de tamaño $k - 2$ del conjunto $\{2, \dots, n - 1\}$, que tiene $n - 2$ elementos. Por lo tanto la cantidad de soluciones factibles es

$$|\mathcal{S}| = \binom{n-2}{k-2}$$

Para darnos una idea de la dimensión exponencial que tiene este problema al considerar todas las posibles combinaciones de pulsos, para una instancia $n = 30$ y $k = 15$ ya hay más de 37 millones de recortes distintos.

Modelamos el problema también en forma de recurrencia, que más adelante servirá para explicar el algoritmo programación dinámica que hicimos con enfoque top-down.

Definimos $g(j, t)$ como el *costo mínimo* de una selección $1 = i_1 < i_2 < \dots < i_t = j$

El mejor modo de llegar al pulso j habiendo conservado t pulsos:

$$g(j, t) = \begin{cases} \infty & \text{si } t > j, \\ c(1, j) & \text{si } t = 2, \\ \min_{t-1 \leq i \leq j-1} \{ g(i, t-1) + c(i, j) \} & \text{si } t \geq 3, \end{cases}$$

La respuesta al problema general entonces es $g(n, k)$.

Vemos caso por caso la recurrencia:

- $t > j$: Es imposible ya que no se puede conservar t pulsos si solo hay j disponibles
- $c(1, j)$: Si solo se toma dos pulsos, uno es el primero y el otro es j
- Caso general: para llegar a j con t pulsos hay que haber venido de algún pulso anterior i con $t - 1$ pulsos y probamos todos los candidatos.

IV. ALGORITMO FUERZA BRUTA

El algoritmo de *Fuerza Bruta* resuelve el problema explorando todas las selecciones posibles de pulsos que cumplen con las restricciones del problema. Como el primer y el último pulso de la grabación deben conservarse obligatoriamente, las decisiones se realizan sobre los $n-2$ pulsos intermedios, cuyas posibilidades son conservarlos o descartarlos para obtener la solución óptima de costo mínimo.

La solución se construye recursivamente, agregando como paso inicial, el primer pulso a *solucion_actual* y luego, se comienza a analizar desde el segundo. En cada llamada recursiva se generan dos ramas en las cuales, en una se agrega el pulso actual i , mientras que en la otra rama no. De esta forma, el algoritmo recorre el árbol de todas las posibles combinaciones de pulsos

Cuando la solución parcial contiene $k-1$ pulsos, se agrega obligatoriamente el último pulso de la grabación, obteniendo una selección de k pulsos. Luego se calcula su costo total y se compara con el de la mejor solución encontrada hasta el momento. Si su costo es menor, pasa a ser la nueva *solucion_mejor*. Si por el contrario, se llega al final de los pulsos intermedios sin haber seleccionado la cantidad necesaria, la rama finaliza sin generar una solución válida.

Algorithm 1: FuerzaBruta(instancia)

```
1 solucionActual  $\leftarrow$  [1];
2 solucionMejor  $\leftarrow$  [];
3  $k \leftarrow$  instancia.k;
4  $i \leftarrow 2$ ;
5 solucionMejor  $\leftarrow$  FB(instancia, solucionActual, solucionMejor,  $k, i$ );
6 return solucionMejor;
```

Algorithm 2: FB(instancia, solucionActual, solucionMejor, k, i)

```
1 if cantidad(solucionActual) =  $k - 1$  then
2   | agregar(solucionActual, instancia.n);
3   | if cantidad(solucionMejor) = 0 then
4   |   | return solucionActual;
5   | end
6   | if costo(solucionActual) < costo(solucionMejor) then
7   |   | solucionMejor  $\leftarrow$  solucionActual;
8   | end
9   | return solucionMejor;
10 end
11 if  $i \geq$  instancia.n then
12 | return solucionMejor;
13 end
14 solucionConI  $\leftarrow$  solucionActual;
15 agregar(solucionConI,  $i$ );
16 solucionMejor  $\leftarrow$  FB(instancia, solucionConI, solucionMejor,  $k, i + 1$ );
17 solucionMejor  $\leftarrow$  FB(instancia, solucionActual, solucionMejor,  $k, i + 1$ );
18 return solucionMejor;
```

i)ANÁLISIS DE COMPLEJIDAD FUERZA BRUTA

El algoritmo recorre un árbol binario ya que para cada pulso intermedio, este se conserva o se descarta. Por eso, la cantidad de nodos crece exponencialmente con n . En el peor caso, la complejidad temporal es del orden de $O(2^n)$.

Además, cada vez que se obtiene una solución completa, se calcula su costo. Una solución contiene k pulsos, por lo que hay $k-1$ saltos a evaluar, y cada salto requiere comparar vectores de d características. Por lo tanto,

calcular el costo de una solución cuesta $O(k \cdot d)$ (aunque en la práctica los vectores de características son cromas de $d = 12$ dimensiones, que sería $O(k)$ en ese caso).

La **complejidad final** es $O(2^n \cdot k \cdot d)$.

V. ALGORITMO BACKTRACKING

El algoritmo de *Backtracking* resuelve el problema de forma muy similar a fuerza bruta. La diferencia es que se incorporan dos *podas* que permiten cortar ramas del árbol antes de seguir explorándolas.

La **poda por factibilidad** calcula cuántos pulsos faltan para completar la solución de tamaño k y cuántos hay disponibles: si faltan más de los que quedan, se descarta porque no producirá una solución válida.

La **poda por optimalidad** compara el costo de la solución parcial con el de la mejor solución encontrada hasta el momento. Como los costos son no negativos, el costo de una solución parcial nunca disminuye al extenderla. Por lo tanto, si el costo es mayor o igual al de *solucion_mejor*, se poda.

En *bt()*, si ya se seleccionaron $k-1$ pulsos, se agrega obligatoriamente el último pulso n y se compara la solución obtenida con la mejor encontrada. Si todavía no se llegó al caso base, se verifican primero las podas y si ninguna se activa, se exploran las dos ramas: una agregando el pulso i y otra sin agregarlo.

Algorithm 1: Backtracking(instancia)

```

1 solucionActual ← [];
2 solucionMejor ← [];
3  $k \leftarrow instancia.k$ ;
4 agregar(solucionActual, 1);
5  $i \leftarrow 2$ ;
6 solucionMejor ← BT(instancia, solucionActual, solucionMejor,  $k, i$ );
7 return solucionMejor;
```

Algorithm 2: BT(instancia, solucionActual, solucionMejor, k, i)

```

1 if cantidad(solucionActual) =  $k - 1$  then
2   agregar(solucionActual, instancia.n);
3   if cantidad(solucionMejor) = 0 then
4     return solucionActual;
5   end
6   if costo(solucionActual) < costo(solucionMejor) then
7     solucionMejor ← solucionActual;
8   end
9   return solucionMejor;
10 end
11 faltanElegir ←  $k - cantidad(solucionActual) - 1$ ;
12 disponibles ←  $instancia.n - i$ ;
13 if faltanElegir > disponibles [Poda por factibilidad] then
14   return solucionMejor;
15 end
16 if cantidad(solucionMejor) ≠ 0 and costo(solucionActual) ≥ costo(solucionMejor) [Poda por
    optimalidad] then
17   return solucionMejor;
18 end
19 if  $i \geq instancia.n$  then
20   return solucionMejor;
21 end
22 solucionConI ← solucionActual;
23 agregar(solucionConI, i);
24 solucionMejor ← BT(instancia, solucionConI, solucionMejor,  $k, i + 1$ );
25 solucionMejor ← BT(instancia, solucionActual, solucionMejor,  $k, i + 1$ );
26 return solucionMejor;
```

ii) ANÁLISIS DE COMPLEJIDAD BACKTRACKING

A diferencia del caso de fuerza bruta donde se exploran todos los caminos posibles, en backtracking se cortan las ramas que ya se sabe que no van a mejorar la solución. Sin embargo, puede ocurrir que la poda no se active nunca por lo que en el peor caso la complejidad será igual que en el caso de fuerza bruta.

La **complejidad final** es $O(2^n \cdot k \cdot d)$.

VI. ALGORITMO PROGRAMACIÓN DINÁMICA

El algoritmo de *Programación Dinámica* aprovecha que distintas ramas pueden llegar al mismo subproblema. La idea es que si se quiere construir una selección que termine en el pulso j habiendo conservado t pulsos, el costo mínimo dependerá finalmente de esos valores (j, t) . Por eso, los resultados de estos subproblemas se guardan en una tabla ('memo') para evitar calcularlos varias veces.

El *caso base* ocurre cuando $t = 2$, ya que la selección está formada únicamente por el primer pulso y el pulso j . Para $t > 2$, se consideran como predecesor todos los pulsos $i < j$ que puedan contener $t - 1$ pulsos y para cada uno se suma el costo que tiene hasta i , más el costo desde i a j , quedándose con el mínimo.

La implementación utiliza un enfoque *top-down* con *memoización*. La función $pd()$ calcula recursivamente cada estado (j, t) y guarda su resultado en $memo[j][t]$. Al mismo tiempo, $padre[j][t]$ almacena el índice i que produjo el menor costo.

Finalmente, $ProgramacionDinamica::resolver()$ calcula el estado (n, k) , que representa el costo mínimo de conservar exactamente k pulsos terminando obligatoriamente en n . Luego, $reconstruir_solucion_pd()$ recorre la tabla padre hacia atrás desde (n, k) hasta llegar al primer pulso y al invertir los índices obtenidos, reconstruye la selección óptima completa.

Algorithm 1: ProgramacionDinamica(instancia)

```

1  $k \leftarrow instancia.k$ ;
2  $n \leftarrow instancia.n$ ;
3  $memo \leftarrow matriz(n + 1, k + 1, -1)$ ;
4  $padre \leftarrow matriz(n + 1, k + 1, -1)$ ;
5  $PD(instancia, n, k, memo, padre)$ ;
6  $solucion \leftarrow Reconstruir(padre, n, k)$ ;
7 return  $solucion$ ;

```

Algorithm 2: PD(instancia, j , t , memo, padre)

```

1 if  $t > j$  then
2   | return  $\infty$ ;
3 end
4 if  $t = 2$  then
5   |  $padre[j][2] \leftarrow 1$ ;
6   | return  $costo(1, j)$ ;
7 end
8 if  $memo[j][t] \geq 0$  then
9   | return  $memo[j][t]$ ;
10 end
11  $res \leftarrow \infty$ ;
12  $mejorPadre \leftarrow -1$ ;
13 for  $i \leftarrow t - 1$  to  $j - 1$  do
14   |  $candidato \leftarrow PD(instancia, i, t - 1, memo, padre) + costo(i, j)$ ;
15   | if  $candidato < res$  then
16     |  $res \leftarrow candidato$ ;
17     |  $mejorPadre \leftarrow i$ ;
18   | end
19 end
20  $memo[j][t] \leftarrow res$ ;
21  $padre[j][t] \leftarrow mejorPadre$ ;
22 return  $res$ ;

```

Algorithm 3: Reconstruir(padre, n , k)

```
1 indices ← [ ];
2 j ← n;
3 t ← k;
4 while t > 1 do
5   agregar(indices, j);
6   j ← padre[j][t];
7   if j < 0 then
8     return [ ];
9   end
10  t ← t - 1;
11 end
12 agregar(indices, 1);
13 invertir(indices);
14 solucion ← [ ];
15 foreach indice en indices do
16   agregar(solucion, indice);
17 end
18 return solucion;
```

iii) ANÁLISIS DE COMPLEJIDAD PROGRAMACIÓN DINÁMICA

El algoritmo utiliza dos tablas: *memo* y *padre*, de tamaño $(n+1) \times (k+1)$. Cada estado (j,t) representa el costo mínimo de seleccionar t pulsos terminando en el pulso j . Por la memoización, cada estado se calcula como máximo una vez. Como existen a lo sumo $n \cdot k$ estados distintos, la cantidad de subproblemas que debe resolver el algoritmo es $O(n \cdot k)$.

Para calcular cada estado (j,t) , se recorren los posibles pulsos anteriores i , desde $t-1$ hasta $j-1$. En el peor caso, este recorrido tiene hasta n iteraciones. Por lo tanto: $O(n \cdot k) \cdot O(n) = O(n^2 \cdot k)$.

En cada iteración también se calcula $costo(i,j)$. Como los vectores de características utilizados tienen dimensión d (asumimos que d es variable):

La **complejidad final** es $O(n^2 \cdot k \cdot d)$.

La reconstrucción de la solución recorre únicamente los k pulsos seleccionados, por lo que cuesta $O(k)$ y no modifica la complejidad total. Respecto de la memoria, las tablas *memo* y *padre* almacenan $n \cdot k$ posiciones cada una. Por lo tanto, la complejidad espacial es $O(n \cdot k)$.

VII. DIFERENCIA DE IMPLEMENTACIONES

Los tres algoritmos fueron implementados primero en C++ y luego se realizaron versiones equivalentes en Python. Se buscó mantener la misma estructura en ambos lenguajes, utilizando los mismos pasos, nombres de funciones y la misma convención de índices. De esta forma, al comparar los tiempos de ejecución, las diferencias observadas pueden atribuirse principalmente al lenguaje.

Una decisión de implementación fue calcular el costo $c(i,j)$ únicamente cuando es necesario, en lugar de almacenar previamente todos los costos posibles. En fuerza bruta y backtracking se decidió pasar la solución parcial por copia en las llamadas recursivas. Esto simplificó el manejo del estado, ya que no fue necesario deshacer los cambios realizados al regresar de una llamada, aunque a cambio se hicieron más copias.

Al trasladar los algoritmos a Python aparecieron algunas diferencias del lenguaje. En C++, pasar una solución por valor genera automáticamente una copia, mientras que en Python los objetos son compartidos, por lo que fue necesario hacer las copias de manera explícita.

Otra diferencia apareció en el manejo de índices. Durante el desarrollo se corrigió un caso en el que podía intentarse acceder a padre[-1]. En C++ es un acceso inválido, mientras que en Python el índice -1 es válido y representa la última posición de una lista. Por lo tanto, el mismo error podía continuar ejecutándose sin producir una falla y devolver un resultado incorrecto.

En cuanto a la dificultad de implementación, backtracking resultó muy cercano a fuerza bruta, ya que mantiene la misma estructura recursiva y agrega únicamente las dos podas. Programación dinámica requirió una implementación más extensa porque, además de definir la recurrencia, fue necesario utilizar las tablas memo y padre y agregar una etapa posterior para reconstruir la solución.

Para la implementación en Python y para algunos programas auxiliares utilizados en los experimentos se utilizó ChatGPT como herramienta de asistencia.

Link a la conversación: https://chatgpt.com/s/cx_6aaa9f0229f481918c62f8d5e02fad99

VIII. INSTRUCCIÓN DE COMPILACIÓN Y EJECUCIÓN

Desde la carpeta TP1/ (cd TP1), el programa en C++ puede compilarse utilizando:

```
make
```

Una instancia puede resolverse indicando el archivo de entrada y el algoritmo deseado. Por ejemplo:

```
./radioedit --instancia input/ejemplo.txt --algoritmo pd
```

La versión en python se ejecuta de manera equivalente:

```
python3 python/main.py --instancia input/ejemplo.txt --algoritmo pd
```

El parámetro --algoritmo puede tomar los valores fb, bt y pd, correspondientes a Fuerza Bruta, Backtracking y Programación Dinámica. En ambos lenguajes la salida contiene los k pulsos seleccionados, numerados desde 1 y en orden creciente.

Las pruebas automáticas se ejecutan mediante:

```
make test
```

Los experimentos se reproducen desde la misma carpeta. Primero se compilan las versiones que cuentan nodos y las que apagan cada poda:

```
make instrumentado ablacion
```

Después se corren los experimentos, todos juntos o de a uno:

```
pyhton3 python/experimentos.py --todos
```

```
pyhton3 python/experimentos.py exp1
pyhton3 python/experimentos.py exp2
pyhton3 python/experimentos.py exp3
pyhton3 python/experimentos.py exp4
```

Las mediciones se guardan en archivos .csv dentro de la carpeta experimentos/resultados_nuevos/, ubicada al lado de TP1/, y los gráficos se regeneran con:

```
pyhton3 python/figura.py --todas
```

Los gráficos se guardan como archivos .svg en la carpeta experimentos/figuras_nuevas/.

IX. METODOLOGÍA DE EXPERIMENTACIÓN

Todas las mediciones se hicieron en la misma computadora: una MacBook Air con procesador Apple M4, 16 GB de memoria y macOS 15.6. El código en C++ se compiló con Apple Clang 17 y optimización -O2, y para Python usamos la versión 3.9.6.

Medimos el tiempo que tarda cada algoritmo en resolver una instancia. Ese tiempo no incluye leer el archivo ni escribir el resultado. Además, en backtracking contamos cuántos nodos del árbol visita, así medir su trabajo, evitando depender del dispositivo con el que hicimos las mediciones

Usamos dos tipos de instancias, cada uno por un motivo distinto:

- Instancias aleatorias, generadas por nosotros. Cada característica es un número al azar entre 0 y 1, con $d = 12$ (como los cromas de una grabación real) y conservando la mitad de los pulsos ($k = n/2$). Como la mayoría de las partes no se repite, casi ningún corte es barato y queríamos observar cómo rendían las podas en esos casos.
- Grabaciones reales: nueve grabaciones de distintos estilos, convertidas en instancias con el programa `extraer.py`, conservando el 75 % de los pulsos.

A la hora de medir los resultados, se probaron sobre 3 instancias aleatorias distintas y cada una se corrió 10 veces y finalmente nos quedamos con la mediana. Si un algoritmo tarda más de 10 segundos en un caso, lo consideramos no viable para ese tamaño y no lo seguimos probando con tamaños mayores.

Un cuidado con backtracking es que la cantidad de nodos que visita cambia mucho de una instancia a otra, aun con el mismo n y k . Por eso nunca sacamos conclusiones a partir de una sola instancia.

X. EXPERIMENTACIÓN Y ANÁLISIS DE RESULTADOS

1. Hasta dónde llega cada algoritmo

En este experimento se buscó observar hasta qué tamaño de instancia resulta viable cada algoritmo. Para ello se aumentó progresivamente n , utilizando $k = \frac{n}{2}$, con instancias aleatorias. Se esperaba que fuerza bruta y backtracking alcanzaran rápidamente tiempos demasiado altos por su comportamiento exponencial, mientras que programación dinámica pudiera trabajar con instancias considerablemente mayores.

Los resultados muestran una diferencia importante entre los tres algoritmos. Fuerza bruta deja de ser viable alrededor de $n = 30$. Por ejemplo, en $n = 28$ ya necesita 6,6 segundos. Backtracking resuelve hasta $n = 300$ y en $n=400$ supera los 10 segundos, mientras que programación dinámica puede resolver $n = 2000$ en 2,4 segundos.

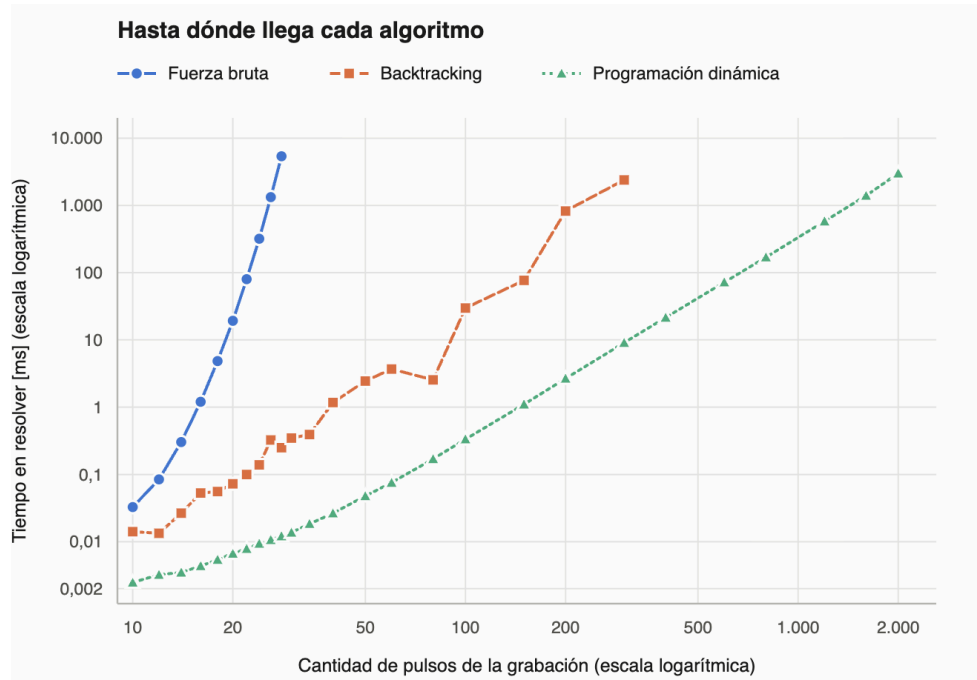


Figura 1: Tiempo que tarda cada algoritmo según la cantidad de pulsos. Los dos ejes están en escala logarítmica: en esta escala, una curva que crece rápidamente permite apreciar con claridad la diferencia de orden entre los algoritmos. Cada serie termina cuando el algoritmo supera los 10 segundos.

2. C++ contra Python

Para responder a la pregunta: ¿cuánto más lento es Python?, queríamos saber si hay un número fijo del estilo ‘Python es tantas veces más lento’ o si además dependía del algoritmo que se esté usando.

Para comprobar esa hipótesis realizamos el siguiente experimento, donde medimos cuántas veces tardaba más correr el mismo algoritmo en Python que en C++ para la misma instancia, y además evaluar cómo era el rendimiento de los 3 algoritmos escritos en Python en función de diferentes tamaños para las instancias. A continuación en el gráfico podemos ver los resultados:

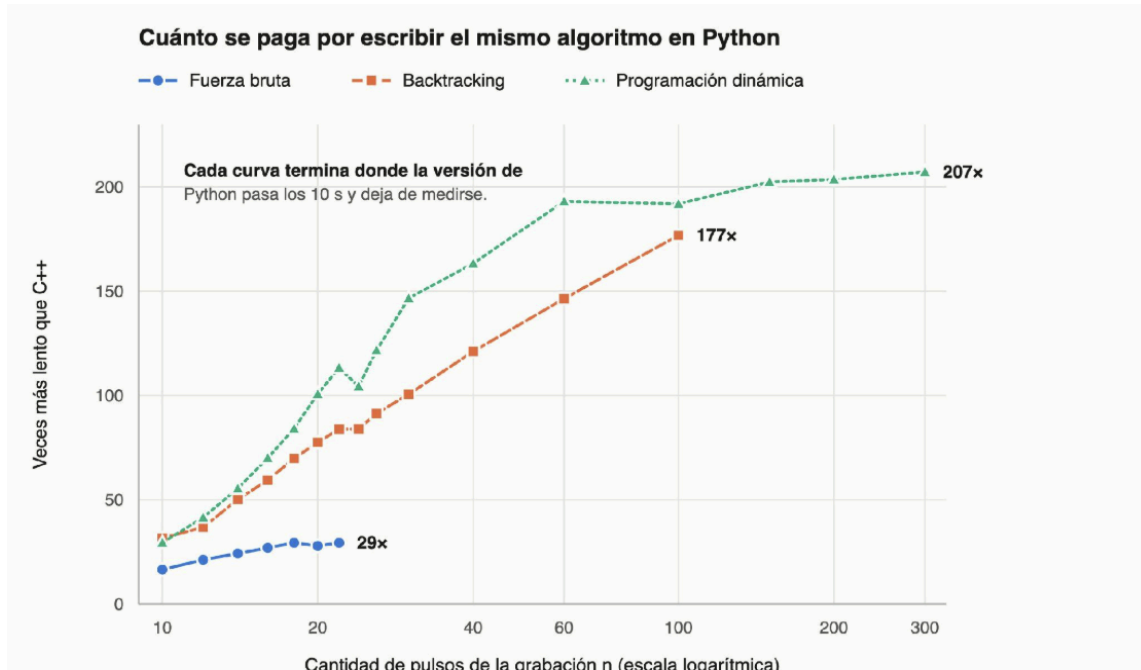


Figura 2: Cuántas veces más lento es cada algoritmo en Python que en C++, según la cantidad de pulsos. Cada curva termina donde la versión de Python pasó los 10 segundos (salvo programación dinámica que terminó la medición (n=300, 1,4 s))

n	FB en C++	FB en Python	BT en C++	BT en Python	PD en C++	PD en Python
10	0,027	0,440	0,012	0,378	0,002	0,063
18	5,80	170,2	0,050	3,47	0,004	0,344
22	104,4	3.065	0,097	8,17	0,006	0,626
26	1.564	—	0,343	31,3	0,008	1,02
60	—	—	3,83	560,5	0,061	11,7
100	—	—	29,6	5.225	0,283	54,3
150	—	—	76,8	—	0,880	178,1
300	—	—	2.304	—	6,85	1.419

Cuadro 1: Tiempo que tarda cada algoritmo (ms) con variables de de entrada: $d = 12$, $k = \frac{n}{2}$ y distintos tamaños n.

Se corrieron 10 veces para cada instancia n y se tomó la mediana de las mediciones del respectivo conjunto de análisis algoritmo/lenguaje. El guión ‘-’ quiere decir que se superó los 10 segundos sin obtener resultado.

El gráfico muestra cuántas veces más lento es Python (en cada algoritmo), mientras que la tabla muestra cuánto tarda efectivamente. La información más destacada de la tabla se podría resumir en que cuánta diferencia podemos observar entre los algoritmos, viendo hasta dónde es que llegan sin superar el tope de 10

segundos: backtracking resuelve $n = 100$ en 30 ms en C++ y 5,2 segundos en Python, mientras que programación dinámica resuelve $n = 300$ en 7 ms en C++ y en 1,4 segundos en Python.

Observando los datos y el gráfico, concluimos que no existe un valor universal que refleje cuánto más lento es Python comparado a C++. Nuestra otra hipótesis era que si además del lenguaje, se deba también a la implementación del algoritmo: obtuvimos que Python llega a ser 29 veces más lento en fuerza bruta, 177 veces en backtracking y 207 veces en programación dinámica.

Además podemos destacar que C++ tiene mejores resultados con instancias más grandes: Fuerza Bruta arranca con que es 17 veces más lento en Python y las otras dos parten de que son alrededor de 30 veces más lentos que su versión en C++ y después se separan cada uno a su manera a medida que aumenta el tamaño de la instancia.

Al ver las mediciones, pudimos comprobar que la hipótesis que se sostiene es que Python es más lento en general, pero la diferencia crece porque C++ se vuelve más eficiente a medida que la instancia crece, tiene tareas fijas que en instancias chicas pesan mucho y en instancias grandes casi no se notan, mientras que Python paga siempre más o menos lo mismo por cada operación.

Esa diferencia inicial, en relación a la velocidad que se corre un algoritmo en Python o en C++, es consistente con las características de ambos lenguajes, ya que C++ es un lenguaje compilado, mientras que Python es interpretado, lo que introduce una sobrecarga adicional en la ejecución.

Lo que claramente se puede ver en los resultados de este experimento es que en Python, fuerza bruta no pasa de $n = 22$ y backtracking no pasa de $n = 100$ cuando se llegaron a los 10 segundos.

3. *Qué aporta cada poda*

Backtracking tiene dos podas, pero la pregunta que queríamos responder es *¿cuánto aporta cada una?*.

Como hipótesis, esperábamos que la de optimalidad hiciera casi todo el trabajo, porque es la única que mira el costo. Nuestro método de experimentación fue que apagamos las podas de a una y comparamos cuántos nodos visita backtracking en cada caso.

Podas activas	Nodos visitados	Nodos evitados	Reducción	Veces que cortó
Ninguna	3.488.871	0	—	—
Sólo factibilidad	2.288.131	1.200.740	1,5 veces	497.420
Sólo optimalidad	1.185	3.487.686	2.944 veces	463
Las dos	845	3.488.026	4.129 veces	57 (fact.) y 297 (opt.)

Cuadro 2: Nodos que visita backtracking según qué podas están activas ($n = 24$, $k = 12$, $d = 12$, instancias aleatorias).

La poda por optimalidad vemos que sola evita el 99,9 % de los nodos visitados. La de factibilidad corta muchas más veces, pero cerca del final del árbol, donde las ramas son chicas y casi no quedan pulsos disponibles.

En cambio la de optimalidad puede dispararse a cualquier altura y cuando lo hace más arriba sobre todo, es capaz de cortar media rama por delante, que pueden llegar a ser muchos los nodos que se estarían ahorrando. Aún así, conviene dejar la poda por factibilidad ya que sumada a la otra, reduce los nodos 1,4 veces más y computacionalmente sólo cuesta comparar dos números (*faltanElegir > disponibles*).

Además descubrimos que con las dos podas apagadas, backtracking visita exactamente la misma cantidad de nodos que fuerza bruta. Es decir, backtracking es fuerza bruta más dos *if*.

4. *¿Qué algoritmo sirve para recortar canciones de verdad?*

En los experimentos anteriores usamos datos inventados por nosotros, pero esta vez queríamos ver qué

algoritmos servían para recortar música de verdad. Entonces lo que hicimos fue darle a cada algoritmo una canción real, le pedimos que la recorte y medimos cuál terminaba a tiempo.

Para experimentar esto, elegimos nueve grabaciones reales de estilos distintos: una persona leyendo en voz alta, drum & bass, dos piezas clásicas, un vals, un ragtime, un folk, un reggaetón y una canción electrónica. Duran desde 15 segundos hasta 3 minutos. Primero convertimos cada grabación en una lista de pulsos con el programa `extraer.py`. Después le pedimos a cada algoritmo que se quede con el 75 % de la grabación/pulsos. Si un algoritmo tardaba más de 10 segundos, lo dábamos por perdido.

Nuestra hipótesis era que esperábamos que fuerza bruta sólo pudiera con la grabación más corta, que backtracking pudiera con algunas y con otras no, y que programación dinámica pudiera con todas.

Aclaración: Las grabaciones se convirtieron previamente a instancias con `extraer.py` (TP1/input/librosa y TP1/input/propias).

Grabación	Duración	Pulsos	Fuerza bruta	Backtracking	Prog. dinámica
Voz hablada	15 s	27	6,1 s	0,16 ms	0,01 ms
Drum & bass	25 s	57	más de 10 s	0,80 ms	0,03 ms
Clásica (Brahms)	46 s	101	más de 10 s	126,2 ms	0,12 ms
Vals	49 s	117	más de 10 s	166,2 ms	0,18 ms
Ragtime	1:11 min	169	más de 10 s	373,2 ms	0,52 ms
Clásica (Tchaikovsky)	2:00 min	212	más de 10 s	7,8 s	1,0 ms
Folk	2:13 min	248	más de 10 s	823,1 ms	1,6 ms
Reggaetón (Bad Bunny)	3:00 min	283	más de 10 s	4,9 s	2,4 ms
Electrónica (Daft Punk)	3:00 min	329	más de 10 s	7,8 s	3,6 ms

Cuadro 3: Cuánto tardó cada algoritmo en recortar cada grabación. Blanco: terminó en menos de 10 segundos. Gris: no terminó a tiempo. Cada tiempo es la mediana de 10 intentos.

Lo que observamos fue que Programación dinámica recortó todas las grabaciones en menos de 4 milisegundos, incluso las canciones de 3 minutos. En la práctica, es instantáneo.

Fuerza bruta sólo pudo con la grabación de 15 segundos, y tardó 6 segundos. Con todas las demás se pasó del límite al intentar probar con todas las combinaciones posibles básicamente.

Backtracking nos sorprendió, terminó con todas, pero tardó muy distinto según la música, no según lo que dura. El folk, de 2 minutos y 13 segundos, lo recortó en menos de un segundo. La pieza de Tchaikovsky, que es más corta, le llevó casi 8 segundos. Y con las canciones de 3 minutos tardó entre 5 y 8 segundos, rozando el límite.

Como conclusión, para recortar una canción, programación dinámica es la opción clara: siempre tarda milisegundos. Backtracking puede llegar a servir, pero en las canciones de 3 minutos llegó a ser unas 2.000 veces más lento (7,8s vs 3,6 ms) y no se puede saber de antemano cuánto va a tardar. Por otro lado, Fuerza bruta no sirve para música real.

XI. CONCLUSIÓN

A partir de la experimentación realizada, se puede concluir que la programación dinámica es el enfoque que presenta un desempeño más adecuado para resolver el problema en instancias de mayor tamaño. La fuerza bruta deja de ser práctica relativamente rápido, mientras que el backtracking presenta un comportamiento más variable y depende en gran medida de las características de cada instancia. En cambio, la programación

dinámica permite trabajar con cantidades de pulsos significativamente mayores, manteniendo tiempos de ejecución razonables. Esto muestra que, a medida que aumenta el tamaño del problema, la elección del algoritmo tiene un impacto fundamental sobre la posibilidad de obtener una solución en un tiempo aceptable.

En cuanto a la implementación, las dos podas utilizadas resultaron especialmente importantes para reducir la cantidad de nodos explorados. En las pruebas realizadas, estas técnicas permitieron que se llegara a reducir alrededor de cuatro mil veces en algunos casos, utilizando únicamente dos comparaciones. Por otra parte, no fue necesario almacenar una tabla con todos los costos posibles, ya que estos pueden calcularse a medida que se necesitan. Esto permite mantener una implementación más eficiente en cuanto al uso de memoria sin afectar el funcionamiento del algoritmo.

La comparación entre los lenguajes también mostró una diferencia considerable de rendimiento. En las pruebas realizadas, C++ obtuvo tiempos de ejecución que fueron varias veces menores que los de Python, con diferencias que aumentaron según el algoritmo y el tamaño de la instancia. Esta diferencia también se reflejó en el tamaño de los problemas que pudieron abordarse: mientras que las implementaciones en Python encontraron rápidamente sus límites, C++ permitió trabajar con instancias considerablemente mayores. Por lo tanto, Python puede resultar útil para probar y validar ideas, mientras que C++ resulta más apropiado cuando se busca una implementación orientada a problemas de mayor dimensión.

Finalmente, los resultados deben considerarse teniendo en cuenta las limitaciones de la experimentación. Las mediciones fueron realizadas sobre una única computadora, por lo que los tiempos pueden variar dependiendo del entorno de ejecución. Además, el comportamiento del backtracking puede cambiar significativamente entre distintas instancias, lo que dificulta establecer conclusiones generales únicamente a partir de los tiempos obtenidos. Aun así, los resultados permiten identificar tendencias claras respecto de la escalabilidad de los algoritmos y de la importancia de las decisiones de implementación. Como posibles mejoras, se plantea comenzar el backtracking utilizando como primera solución el mejor corte único encontrado, mantener el costo acumulado para evitar cálculos repetidos y continuar optimizando la implementación de la programación dinámica. En conjunto, la experimentación muestra que la combinación entre una elección adecuada del algoritmo, técnicas de poda y una implementación eficiente resulta fundamental para abordar instancias de mayor tamaño.